Fall 2026 CIS*3210
Assignment 2: UDP Socket Programming

Weight: 6%

Due on: October 19, 2026, at 11:59 PM

Purpose

Build two Python programs that exchange numbered UDP probe messages: a client that sends requests and measures replies, and a server that receives requests and responds. Also write and submit test_invalid.py to test an invalid request. Capture one complete ten-probe run in Wireshark and explain how its datagrams correspond to the Python messages.

What you will practise

- Create IPv4 UDP sockets and exchange datagrams with `sendto()` and `recvfrom()`.
- Use server and client IP addresses and ports correctly without a TCP connection.
- Apply a one second reply deadline, match responses to requests, and calculate round trip times.
- Identify request and response payloads, ports, and timing in a Wireshark capture.

Programs and protocol

Submit server.py, client.py, and test_invalid.py. Use Python 3 and only its standard library. All three scripts must create a socket using `socket.AF_INET` and `socket.SOCK_DGRAM`. Do not use TCP, ICMP, raw sockets, a networking framework, or an external Python package. Each request or response is one UDP datagram containing UTF-8 text, without a trailing newline.

Code comments. Include enough meaningful comments in server.py, client.py, and test_invalid.py to explain the main steps: socket setup, request validation, deliberate reply omissions, probe sequence, deadline and reply matching, round-trip timing, summary calculations, cleanup, and the purpose and expected result of the invalid-request test. Add a short description of each function. Explain why non-obvious steps are needed; do not merely repeat every statement in words. Comments are assessed under Communication and Submission Quality.

Direction	Exact text pattern	Example
Client to server	PING <n>	PING 3
Server to client	PONG <n>	PONG 3
Server error reply	ERROR invalid request	ERROR invalid request

Replace <n> with a positive integer from 1 to 9999 written using ASCII digits 0-9, without a leading zero. Use exactly one space after PING or PONG and no other characters or spaces. Commands are case sensitive. For example, PING 3 is valid; ping 3, PING 0, PING 03, PING 3 extra, and an empty datagram are invalid requests. The client sends only PING 1 through PING 10. A 1024 byte receive buffer is sufficient for these messages.

Server requirements

- **Start and bind**. Log in to linux-01.socs.uoguelph.ca and run `python3 server.py <primary_port>` there. Replace <primary_port> with the primary port assigned to you by email. Bind one IPv4 UDP socket to ("", primary_port), so it can receive datagrams addressed to the SoCS host. Print the port when ready. Do not use your secondary port or another student's port for this assignment.
- Receive repeatedly. Call recvfrom() in a loop and keep the server running until you stop it with Ctrl+C. Use the returned sender IP address and source port for replies.
- **Validate requests**. Decode each datagram as UTF-8 and check the exact PING format above. If decoding fails or the request is invalid, send exactly ERROR invalid request to that sender, log the error, and continue receiving other requests. Invalid requests must not trigger a PONG or the omission rule.
- **Reply or omit**. For a valid PING <n>, if n is divisible by 5, deliberately send no response datagram. This rule depends on the sequence number, not on how many datagrams the server has received. For every other valid request, send exactly PONG <n> to the address returned by `recvfrom()`.
- **Log and close**. For each datagram, print the sender IP address and source port, the received text when decodable, and whether the server sent a PONG, sent an ERROR, or deliberately omitted a reply. Close the socket when the server ends. The task does not require threads or simultaneous processing of clients.

Client requirements

- **Start and address**. Run the client on your own computer, outside the SSH session: python3 client.py linux-01.socs.uoguelph.ca <primary_port>. Use the same assigned primary port as the server. Create one IPv4 UDP socket for the entire run. Let the operating system choose the client source port; do not bind the client to your assigned server port. You may pass the hostname directly to sendto(); explicit hostname resolution and reply source-address checking are not required.
- **Send ten probes**. Send PING 1, PING 2, ..., PING 10 in that order, exactly once each. Send the next probe only after the previous probe receives a matching PONG, receives ERROR invalid request, or reaches its one-second deadline. Do not retry a request. A short pause between probes is optional.
- **Match replies**. For each probe, wait up to one second measured from immediately before sending it. Count a success only when the received text is exactly PONG <n>, where <n> is the current sequence number. Ignore late or mismatched PONGs and other unexpected or undecodable payloads, and keep waiting until the original deadline. Do not restart the one-second period after ignoring a datagram. If the exact ERROR invalid request response arrives, handle it as described next. This assignment uses one replying server and does not require checking the source IP address or port.
- **Show outcomes**. Print each sequence number and either its successful round-trip time in milliseconds or TIMEOUT when no matching PONG arrives by the deadline. If the exact response ERROR invalid request arrives, display ERROR, count that probe as having no matching PONG, and continue to the next number. Use a monotonic clock for timing; measure from immediately before `sendto()` to receipt of the matching PONG.
- **Summarize and exit**. After probe 10, print requests sent, matching PONG replies, requests without a matching PONG, and the percentage without a matching PONG. Print minimum, average, and maximum round trip time using only successful probes; use N/A for all three if there were none. Handle a bad host or port and socket failures with a clear message and close the socket.

Expected run and checks

SSH terminal on your computer: `ssh YOUR_USERNAME@linux-01.socs.uoguelph.ca`
On linux-01: `python3 server.py <primary_port>`
Local terminal on your computer: `python3 client.py linux-01.socs.uoguelph.ca <primary_port>`

When working off campus, connect to the University VPN before using SSH to access linux-01.socs.uoguelph.ca. Keep the VPN connected while editing or running the server remotely and testing your local client. Replace YOUR_USERNAME with your SoCS username and <primary_port> with the number from your assigned ports email. On macOS or Linux, use Terminal; on Windows, use PowerShell for ssh. Check the host key against the usual SoCS instructions before accepting it. Enter your password when prompted; it will not appear as you type. Place server.py in your SoCS account using an editor in the SSH session or a permitted file transfer method. Write and run client.py on your own computer. Keep the SSH session and server running while testing. In a clean run, the server receives ten PING datagrams, sends eight PONG datagrams, and deliberately sends nothing for PING 5 and PING 10. The client should report 10 sent, 8 matching replies, 2 without a matching reply, and 20% without a matching reply. Round trip times vary.

Invalid-request test. Write and submit test_invalid.py. Run it on your own computer with python3 test_invalid.py linux-01.socs.uoguelph.ca <primary_port>, while server.py is running on linux-01. Use your assigned primary port. The test script must send at least one invalid request, such as PING 0 or PING 3 extra, using sendto(), read the response using recvfrom(), and display the request sent, the expected response ERROR invalid request, and the actual response. If no response arrives within one second, display TIMEOUT. Close the test socket when finished. Confirm that the server remains running by running client.py afterward without restarting the server. Run this test separately from the required ten-probe Wireshark capture. Include the test script in your named submission folder and record its command, invalid request, expected response, and observed result in README.txt. A timeout in the ten-probe run does not prove UDP packet loss: the server intentionally sends no reply for two requests.

Wireshark capture

- Complete the required Wireshark capture on campus, with the VPN disconnected. Off-campus VPN use can hide the original PING and PONG datagrams inside encrypted tunnel traffic on the physical Wi-Fi or Ethernet interface. A VPN interface may expose the original datagrams on some systems, but this assignment requires an on-campus capture so interface differences do not prevent completion. Plan time on campus for the capture; you may analyze the saved capture later off campus.

  Capture one clean run in Wireshark on your own computer. Select the active Wi-Fi or Ethernet interface carrying traffic to linux-01; do not select loopback. Before starting the capture, enter udp port <primary_port> in the capture-filter field for the selected interface, replacing <primary_port> with your assigned numeric port. This records UDP datagrams with that source or destination port. Start capture before launching the local client and stop after probe 10. The server runs on linux-01 and is started through SSH; the probes and replies travel directly as UDP datagrams between your client and server. Save all packets recorded with this capture filter, without editing packet contents or deleting packets, as A2_<StudentNumber>_<LastName>.pcapng.
- After recording, apply the display filter `udp.port == <primary_port>`, replacing the placeholder with your assigned number. Inspect the packet list, UDP source and destination ports, and packet bytes or Data field to read the PING and PONG payloads. A clean, successful run should show 18 UDP datagrams: ten requests and eight responses. The capture filter udp port <primary_port> controls which packets are recorded. The display filter udp.port == <primary_port> controls only which recorded packets are shown. SSH uses TCP and is excluded by the capture filter. Keep the capture short, check that it reopens, and avoid including unrelated personal traffic.

Analysis questions

Write a PDF report of about 1 to 3 pages, named A2_<StudentNumber>_<LastName>.pdf. Include at least one readable screenshot of the filtered packet list. Include a packet details or bytes screenshot if the payload is not readable in the list. Answer all four questions with frame numbers from your own capture:

1. Choose one PING and its matching PONG. Give both frame numbers, the source and destination IP addresses and UDP ports for each, and their exact payload text. Explain how the server knows the client address and port for its reply.
2. Find PING 5 and PING 10. Give their frame numbers and state whether PONG 5 or PONG 10 appears. Explain why they have no reply in this assignment, and why their absence alone is not evidence that the network lost the datagrams.
3. For one successful sequence, report the difference between the PING and PONG packet times in Wireshark and the round-trip time printed by your client. Explain briefly why the values can differ.
4. Identify the server port and client source port. Explain who chose each. State why this capture contains no TCP handshake or TCP acknowledgments for this application.

Submission requirements

Submit exactly two items together to the designated Assignment 2 Dropbox on CourseLink: one PDF report and one ZIP archive. The ZIP must contain one named folder with server.py, client.py, test_invalid.py, original screenshots, original live capture, and README. Use the prefix A2 in the naming convention below, including for this Assignment 2 submission. Replace StudentNumber and LastName with your information and do not include the angle brackets.

- Name the PDF report A2_<StudentNumber>_<LastName>.pdf. Organize it by Analysis Questions 1 through 4 in the order shown above. Keep those question numbers in the report.
- Place each screenshot next to the answer it supports. Frame numbers, payload text, ports, addresses, and relevant field values must be readable. Include the client output needed to support your timing comparison and summary results. Save the original screenshot image files in the screenshots subfolder inside your submission folder. Use clear names such as filtered_packets.png, ping_pong_details.png, and client_output.png. Include all screenshots used in your PDF, including any additional payload views needed to make the evidence readable.
- Prepare the report using Microsoft Word or another word processor and export it to PDF. Do not submit a scanned handwritten report or an image-only PDF.
- Name the original live-capture file A2_<StudentNumber>_<LastName>.pcapng. Use the capture filter specified above. If unrelated or sensitive traffic is recorded, repeat the controlled capture with the correct filter; do not edit packet contents or delete packets from the saved capture. Verify that the file opens in Wireshark and contains your ten-probe exchange. Place this file directly inside the named submission folder; it is submitted inside the ZIP.
- Create one folder named A2_<StudentNumber>_<LastName>. Place server.py, client.py, test_invalid.py, README.txt, and A2_<StudentNumber>_<LastName>.pcapng directly inside it. Create a screenshots subfolder and place your screenshot image files there. Include any additional scripts or data required to run your programs and document them in README.txt. Compress the entire named folder into A2_<StudentNumber>_<LastName>.zip. The ZIP must contain this single enclosing folder. Upload the PDF separately alongside the ZIP.

Required folder and upload structure

Location	Required item
CourseLink upload 1	A2_<StudentNumber>_<LastName>.pdf
CourseLink upload 2	A2_<StudentNumber>_<LastName>.zip
Inside ZIP root	A2_<StudentNumber>_<LastName>/
Inside the named folder	server.py
Inside the named folder	client.py
Inside the named folder	test_invalid.py
Inside the named folder	README.txt
Inside the named folder	A2_<StudentNumber>_<LastName>.pcapng
Inside the named folder	screenshots/
Inside screenshots	Original screenshot image files used in the report

Example for student number 1234567 and last name Smith: upload only A2_1234567_Smith.pdf and A2_1234567_Smith.zip. Inside the ZIP, the folder A2_1234567_Smith contains the three Python scripts (server.py, client.py, and test_invalid.py), README.txt, A2_1234567_Smith.pcapng, and the screenshots subfolder.

- In README.txt, state the Python versions on both computers, your local operating system, the exact SSH, server, client, and test_invalid.py commands, your assigned primary port, the invalid request sent, its expected response, the observed response or timeout, and whether the server continued to run after the test, and any known limitations. Identify any permitted reference materials you used. AI tools are prohibited, as stated in Academic Integrity. Do not include passwords.
- Before uploading, extract your ZIP into a fresh folder and check that all three submitted scripts run using the README commands, the ZIP can be extracted, and the PDF, capture, and screenshot files open correctly. Only the most recent submission will be graded, and it must contain both the PDF report and the ZIP. If you resubmit either item, include both items again.
- Zero-mark policy. Your entire assignment will receive 0/100 if any required script (server.py, client.py, or test_invalid.py) does not run, contains a syntax error, or terminates with an unhandled runtime error when run using the required commands. A missing PDF, ZIP, required script, or capture file also results in 0/100. Submitting unrelated files, files for another assignment, files of the wrong required type, or corrupt files also results in 0/100. Check both uploads and all ZIP contents before submitting. These rules override the rubric.
- The required ERROR invalid request response, the intentional TIMEOUT outcomes for probes 5 and 10, and normal termination with Ctrl+C are expected behavior and do not trigger the zero-mark policy. Incorrect filenames or folder structure alone are assessed under Communication and Submission Quality. If your submission passes the execution and file checks, incorrect answers or program results, incomplete features, comments, and presentation are assessed using the rubric.

Marking scheme and rubric

This assignment is marked out of 100 using the rubric below. Your work will be assessed against the Excellent, Good, and Unsatisfactory descriptions. Marks within each range depend on the completeness and accuracy of your work.

The zero-mark policy in Submission requirements takes precedence over the rubric. If that policy applies, your entire assignment will receive 0/100.

The Server and Client components assess the individual programs. Protocol and deployment assesses how the programs work together across your computer and the SoCS server. A specific error assessed under an individual program component will not receive an additional deduction under Protocol and deployment solely because it affects the exchange.

A missing or non-running test_invalid.py triggers the zero-mark policy. If your submission passes the execution and file checks, the script's required test behavior, comments, and documentation are assessed under Communication and Submission Quality. The server's handling of invalid requests is assessed under Server request validation. Filename errors alone affect only Communication and Submission Quality.

Component	Marks
Server	25
Client	30
Protocol and deployment	10
Wireshark and analysis	25
Communication and Submission Quality	10
Total	100

Detailed rubric measures

If your submission passes the execution and file checks, your mark for each component will depend on the performance level demonstrated and the completeness and accuracy of your work. The three ranges do not overlap and cover every integer mark from zero to the component maximum. The zero-mark policy takes precedence over all ranges below. Naming errors affect only Communication and Submission Quality.

Component and maximum marks	**Excellent**	**Good**	**Unsatisfactory**
Server socket and operation 5 marks	4–5 marks Runs on linux-01; correctly creates and binds the UDP socket to the assigned primary port; receives repeatedly, prints readiness, and closes cleanly.	2–3 marks Core server runs and receives requests, but readiness output or cleanup is incomplete.	0–1 marks Code executes, but the required receive loop, readiness message, or cleanup is largely missing or incorrect.

Component and maximum marks	Excellent	Good	Unsatisfactory
Server request validation 7 marks	6–7 marks Checks exact format, sequence range, spacing, case, leading zeros, and UTF-8; sends the exact ERROR reply and continues after invalid requests.	3–5 marks Rejects most invalid requests and continues, but one or more validation cases or ERROR details are incorrect.	0–2 marks Code executes, but validation is largely absent or incorrect; malformed requests are accepted or receive unsuitable replies.
Server replies and omissions 8 marks	7–8 marks Sends the exact matching PONG to the sender for valid nonmultiples of 5; deliberately omits replies for multiples of 5 using the sequence number.	4–6 marks Replies mostly work, but a payload, reply address, or omission-rule case is incorrect.	0–3 marks Matching replies or deliberate omissions are largely incorrect or absent; the exchange is unreliable.
Server logging 5 marks	4–5 marks Logs sender IP and source port, decodable request text, and the correct PONG, ERROR, or omission outcome for every datagram.	2–3 marks Useful logs are present, but required details or some outcomes are missing or inaccurate.	0–1 marks Logs are absent or too incomplete or inaccurate to follow requests and outcomes.
Client probe sequence 6 marks	5–6 marks Uses one socket; sends PING 1–10 once each in order; waits for each outcome before advancing and does not retry.	3–4 marks Most probes are correct, but socket reuse, ordering, waiting, or no-retry behavior has a limited error.	0–2 marks Probe sequence or sending behavior is largely incorrect or absent; required ten-probe run is not completed.
Client deadline and reply matching 10 marks	8–10 marks Uses one one-second deadline from before sending; checks the exact PONG payload and current sequence; ignores mismatches without extending the deadline; handles ERROR.	4–7 marks Timeout and reply matching mostly work, but at least one required payload, sequence, deadline, or ERROR case is incorrect.	0–3 marks Deadline or matching behavior is largely incorrect or absent; invalid replies are accepted or waiting can continue beyond the required limit.
Client round trip timing 6 marks	5–6 marks Uses a monotonic clock and correct send and receipt endpoints; displays successful RTTs in milliseconds.	3–4 marks Measures useful RTTs, but the clock, endpoints, units, or displayed results have a limited error.	0–2 marks Timing is missing or substantially incorrect; reported values do not represent the required RTT.
Client summary and errors 8 marks	7–8 marks Correct counts and unanswered percentage; successful-only minimum, mean, and maximum; N/A with no successes; clear failures and socket closure.	4–6 marks Summary is mostly correct, but some statistics, the no-success case, failure messages, or cleanup are incomplete.	0–3 marks Summary or error handling is largely absent or incorrect; counts or statistics cannot be trusted.

Component and maximum marks	Excellent	Good	Unsatisfactory
Protocol and deployment 10 marks	8–10 marks The submitted local client and SoCS server operate together using the documented commands and assigned primary port; a complete ten-probe run demonstrates the required exchange.	4–7 marks Both programs execute and communicate across the required computers, but an integration or configuration issue affects the exchange or requires a correction to the documented setup.	0–3 marks Both programs execute, but the required operation across the local computer and SoCS server is substantially incorrect or not demonstrated.
Capture and readable evidence 7 marks	6–7 marks Capture opens and contains the complete on-campus ten-probe exchange; the filtered packet list and supporting client output are readable. Payloads are readable in the packet-list screenshot or in additional packet-details/bytes screenshots.	3–5 marks Capture is usable, but the exchange or supporting screenshots are incomplete or partly unreadable.	0–2 marks The submitted capture opens, but the recorded exchange or supporting screenshots are too incomplete or unreadable to verify the required evidence.
Analysis Question 1 8 marks	7–8 marks Correct matching frame numbers, both directions of IP addresses and UDP ports, exact payloads, and explanation of recvfrom() reply addressing.	4–6 marks Identifies a valid pair and most fields, but some values, evidence, or reply-address explanation are incomplete or inaccurate.	0–3 marks Pair or field identification is largely wrong or absent; explanation lacks the required addressing concept.
Analysis Question 2 5 marks	4–5 marks Correct PING 5 and 10 frames, reply checks, and explanation of intentional omission versus actual network loss.	2–3 marks Most evidence and explanation are correct, but a frame, reply check, or distinction is incomplete.	0–1 marks Evidence is largely absent or wrong, or omitted replies are incorrectly treated as proof of network loss.
Analysis Questions 3 and 4 5 marks	4–5 marks Supports Wireshark versus client timing comparison, explains possible differences, identifies who chooses each port, and explains the absence of TCP setup and acknowledgments.	2–3 marks Answers most parts correctly, but comparison evidence or one explanation is incomplete or inaccurate.	0–1 marks Most parts are missing or incorrect; timing comparison or UDP and port explanations are unsupported.

Component and maximum marks	Excellent	Good	Unsatisfactory
Communication and Submission Quality 10 marks	8–10 marks Correct PDF, ZIP, and folder names and required file locations. test_invalid.py sends an invalid request to the assigned server and port, receives and displays the actual response, shows the request and expected ERROR invalid request response, handles a one-second timeout, and closes its socket. README contains all required details, including the test result and whether the server continued running. All three scripts have useful comments; the report is clear and organized.	4–7 marks Required scripts and files pass the execution and file checks, but test-script behavior, naming, folder structure, README details, comments, screenshots, or report clarity have limited errors or omissions.	0–3 marks Required scripts and files pass the execution and file checks, but test-script behavior, naming, organization, README details, comments, screenshots, or report quality have substantial deficiencies.

Total available marks: 100. The zero-mark policy overrides this rubric. If your submission passes the execution and file checks, you will receive zero for an individual component if you provide no correct, usable evidence for that component.

Academic Integrity

Your submissions, including both code and reports, will be examined to ensure compliance with academic integrity standards.

This assignment is an **individual** assignment, not a collaborative project. You must complete all parts of the assignment independently.

All work submitted must be your own. You are not permitted to use AI tools, such as ChatGPT, GitHub Copilot, or similar tools, or any code-generation tools to complete this assignment.

You may consult lecture materials, textbooks, and lab examples, but copying or generating code through external assistance will be considered academic misconduct and reported in accordance with university policies.

If you are unsure whether a resource is permitted, please contact the instructor or a teaching assistant before proceeding. You must review the Academic Integrity section of the course outline and the University of Guelph Academic Integrity instructions.